Combined Report

This file combines the previous `report1.md` (pylint/code-quality phase) and `report1.3.md` (white-box testing phase).

Part 1: Code Quality (from report1.md)

Money-Poly Pylint Improvement Report

Scope

Performed iterative code-quality improvements on the Money-Poly Python project using `pylint`, with small category-focused commits.

Baseline (Initial Analysis)

- Initial pylint score: **9.08/10**
- Total issues: **60**

Categorized Findings

- Unused imports / variables:
  - `W0611` (unused-import) x5
  - `W0612` (unused-variable) x1
- Naming convention issues:
  - None reported by pylint in baseline run.
- Formatting / style issues:
  - `C0301` (line-too-long) x24
  - `C0304` (missing-final-newline) x2
  - `C0325` (superfluous-parens) x2
  - `C0121` (singleton-comparison) x1
  - `R1705` (no-else-return) x1
  - `R1723` (no-else-break) x1
  - `W0702` (bare-except) x1
  - `W1309` (f-string-without-interpolation) x1
- Code complexity / structure:
  - `R0902` (too-many-instance-attributes) x3
  - `R0912` (too-many-branches) x1
  - `R0913` (too-many-arguments) x1
  - `R0917` (too-many-positional-arguments) x1
  - `W0201` (attribute-defined-outside-init) x1
- Missing docstrings:
  - `C0114` (missing-module-docstring) x10
  - `C0115` (missing-class-docstring) x2
  - `C0116` (missing-function-docstring) x2

Iterative Changes

Iteration 1

- Commit: `e9fc220`
- Message: `Iteration 1: Removed unused imports and variables`
- Focus: Unused imports / variables only
- Changes:
  - Removed unused imports in `bank.py`, `dice.py`, `game.py`, `player.py`
  - Removed unused local variable in `player.py`
- Score after iteration: **9.20/10**

Iteration 2

- Commit: `f702a4e`
- Message: `Iteration 2: Fixed pylint style and formatting issues`
- Focus: Formatting and style only
- Changes:
  - Reformatted long card definitions in `cards.py`
  - Fixed singleton comparison in `board.py`
  - Replaced bare `except` in `ui.py`
  - Removed unnecessary `else`/`elif` and superfluous parentheses in `game.py` and `property.py`
  - Removed non-interpolated f-string use in `game.py`
- Score after iteration: **9.68/10**

Iteration 3

- Commit: `359981d`
- Message: `Iteration 3: Added missing module, class, and function docstrings`
- Focus: Docstrings only
- Changes:
  - Added module docstrings across all modules
  - Added missing class/function docstrings (`main.py`, `bank.py`, `property.py`)
- Score after iteration: **9.89/10**

Iteration 4

- Commit: `6b13327`
- Message: `Iteration 4: Simplified complexity and reduced excess class state`
- Focus: Complexity/state cleanup and small refactors
- Changes:
  - Reduced excessive instance attributes:

        - Removed unused player elimination field
        - Consolidated game decks into a dictionary and removed redundant runtime flag
        - Simplified property state by computing mortgage value dynamically
    - Reduced branching in card handling by introducing card-action handler methods
    - Fixed `attribute-defined-outside-init` in `dice.py`
    - Simplified `Property` constructor argument shape and updated board property definitions
- Score after iteration: **10.00/10**

Verification Per Iteration

- Re-ran `pylint` after each iteration.
- Performed runtime sanity checks using:
    - `python -m compileall -q .`
    - lightweight import/instantiation smoke checks.

Final Result

- Initial pylint score: **9.08/10**
- Final pylint score: **10.00/10**
- Major warnings remaining: **None**
- Core game flow preserved; no architecture rewrite performed.

Part 2: White-Box Testing (from report1.3.md)

White-Box Testing Report (Subpart 1.3)

1. Objective

Performed white-box testing for the Money-Poly Python project using `pytest` and branch coverage tooling, then fixed all logic defects exposed by tests.

2. Step 1: White-Box Code Analysis

Module-Level Decision/Path Analysis

- `main.py`
    - Branches: normal game start, `KeyboardInterrupt` handling, `ValueError` handling, `__main__` entrypoint branch.
- `moneypoly/bank.py`
    - Branches: `pay_out` non-positive / insufficient / success; `give_loan` non-positive / positive.

  - Loop/aggregation path: `total_loans_issued` generator expression.
- `moneypoly/cards.py`
  - Branches: `draw` empty/non-empty; `peek` empty/non-empty.
  - Cycle path: index modulo deck size.
- `moneypoly/dice.py`
  - Branches: doubles streak increment/reset, doubles note formatting.
- `moneypoly/property.py`
  - Branches: rent normal/mortgaged/full-group bonus; mortgage/unmortgage paths.
  - Group logic branches: add duplicate/not duplicate, all-owned true/false, owner count include/skip.
- `moneypoly/player.py`
  - Branches: add/deduct validation, bankrupt true/false, move pass-go/land-go/no-go, property add/remove list checks.
- `moneypoly/board.py`
  - Branches: tile special/property/blank; purchasable false/false/false/true branches.
- `moneypoly/ui.py`
  - Branches: jailed/non-jailed display, properties present/absent, input valid/invalid/EOF, confirm true/false.
- `moneypoly/game.py`
  - Major control-flow branches:
    * `play_turn`: jailed flow, triple-doubles jail flow, doubles extra-turn flow, normal turn advance.
    * `_move_and_resolve`: all tile types (`go_to_jail`, taxes, free parking, chance, community chest, railroad/property with and without property object, blank).
    * `_handle_property_tile`: buy/auction/skip/owned/rent paths.
    * Economic methods: buy, pay rent, mortgage, unmortgage, trade success/failure branches.
    * Auction loop: pass/low-bid/over-budget/valid-bid + winner/no-winner endings.
    * Jail handler: card use, voluntary fine payment, no-action, mandatory release.
    * Card application: none/unknown/each supported action and helper branches.
    * Bankruptcy elimination with and without list membership and index reset case.
    * Winner selection empty/non-empty.
    * Run loop branches (turn loop, single-player break, no-player ending).
    * Interactive/menu branches for all choices and validation paths.

Key Variable States Tested

- Money edge states: zero, insufficient, exact price, high values.

- Ownership states: unowned/owned-by-self/owned-by-other, mortgaged/unmortgaged.
- Turn/jail states: jailed, jail turn progression, release flows.
- Deck and tile states: empty deck, cyclic deck, all tile categories.

3. Step 2 + Step 3: Test Case Design and Implementation

All tests were implemented in `moneypoly/tests/` and each test targets explicit branches.

`test_config_constants_are_accessible`

- Tests: config constants import and value sanity.
- Branch/path: module execution path for config.
- Why needed: ensures base constants are accessible for all modules.

`test_bank_collect_and_get_balance`

- Tests: positive collect and current balance retrieval.
- Branch/path: standard collect path.
- Why needed: validates bank state mutation.

`test_bank_pay_out_non_positive_and_valid_and_insufficient`

- Tests: payout for zero/negative, normal payout, and insufficient-funds exception.
- Branch/path: all `pay_out` branches.
- Why needed: verifies safe fund handling.

`test_bank_give_loan_and_loan_stats`

- Tests: ignored zero loan, issued positive loan, loan stats and summary output.
- Branch/path: both `give_loan` branches.
- Why needed: confirms loan accounting logic.

`test_carddeck_draw_peek_cycle_and_empty`

- Tests: empty deck draw/peek, normal draw, cyclic draw, shuffle reset.
- Branch/path: empty/non-empty draw and peek branches.
- Why needed: validates deterministic card flow.

`test_dice_roll_updates_streak_and_description`

- Tests: doubles and non-doubles rolls and string formatting.
- Branch/path: streak increment/reset and description branch.
- Why needed: core turn movement relies on dice correctness.

`test_dice_reset_clears_state`

- Tests: explicit state reset.
- Branch/path: reset execution path.
- Why needed: prevents stale dice state.

`test_current_player_and_advance_turn_rotation`

- Tests: turn index rotation and turn counter.
- Branch/path: wrap-around turn progression.
- Why needed: validates turn scheduler behavior.

`test_play_turn_when_player_is_in_jail`

- Tests: jailed player path calls jail handler and advances turn.
- Branch/path: early return jail branch.
- Why needed: jailed flow is distinct and must skip normal movement.

`test_play_turn_three_doubles_sends_to_jail`

- Tests: triple doubles sends player to jail.
- Branch/path: `doubles_streak >= 3` branch.
- Why needed: enforces game rule for repeated doubles.

`test_play_turn_doubles_grants_extra_turn`

- Tests: doubles gives extra roll.
- Branch/path: `is_doubles()` true branch.
- Why needed: verifies extra-turn logic.

`test_play_turn_non_doubles_advances_turn`

- Tests: non-doubles advances turn.
- Branch/path: `is_doubles()` false branch.
- Why needed: normal turn completion behavior.

`test_move_and_resolve_tile_branches`

- Tests: all tile categories and action dispatch.
- Branch/path: full `_move_and_resolve` if/elif chain.
- Why needed: this is the primary board-resolution engine.

`test_move_and_resolve_property_lookup_none_branch`

- Tests: railroad/property tiles with missing property object.
- Branch/path: nested `if prop is not None` false paths.
- Why needed: protects against null lookups.

`test_handle_property_tile_branches`

- Tests: unowned buy/auction/skip, owned-by-self, owned-by-other.
- Branch/path: all `_handle_property_tile` branches.
- Why needed: critical ownership decision logic.

`test_buy_property_branches_and_exact_balance_case`

- Tests: insufficient funds and exact-balance purchase.
- Branch/path: buy failure/success branches.
- Why needed: exact-balance is a common boundary case.

`test_pay_rent_branches_and_transfer_to_owner`

- Tests: mortgaged/no-owner/normal rent payment and owner credit.
- Branch/path: all `pay_rent` branches.
- Why needed: verifies economic correctness during rent events.

`test_mortgage_property_branches`

- Tests: not-owner, already-mortgaged, success mortgage.
- Branch/path: all `mortgage_property` branches.
- Why needed: validates risk-management actions.

`test_unmortgage_property_branches`

- Tests: not-owner, not-mortgaged, cannot afford, success.
- Branch/path: all `unmortgage_property` branches.
- Why needed: ensures debt redemption constraints.

`test_trade_branches_and_cash_transfer_to_seller`

- Tests: invalid ownership, buyer cannot afford, successful trade.
- Branch/path: all trade branches.
- Why needed: prevents invalid trades and checks money transfer integrity.

`test_auction_property_with_winner`

- Tests: pass, too-low bid, unaffordable bid, valid winner.
- Branch/path: per-bid branch checks and winner branch.
- Why needed: auction path has multiple branch conditions.

`test_auction_property_no_bids`

- Tests: all players pass.
- Branch/path: no-winner auction ending branch.
- Why needed: confirms auction fallback behavior.

`test_handle_jail_turn_use_card`

- Tests: uses jail-free card and continues move.
- Branch/path: jailed card-use branch.
- Why needed: validates alternate jail release mechanism.

`test_handle_jail_turn_pay_fine_branch`

- Tests: declines card, pays fine, exits jail.
- Branch/path: voluntary fine branch.
- Why needed: verifies payment path and jail state reset.

`test_handle_jail_turn_no_action_and_mandatory_release`

- Tests: no action increments turn; third turn forces release.
- Branch/path: no-action and mandatory-release branches.
- Why needed: covers jail turn progression boundaries.

`test_apply_card_none_and_unknown_action`

- Tests: `None` card and unknown action.
- Branch/path: `_apply_card` early-return and no-handler branch.
- Why needed: defensive behavior for malformed inputs.

`test_apply_card_known_actions`

- Tests: collect, pay, jail, jail_free, move_to with property and non-property outcomes.
- Branch/path: handler dispatch and `_handle_move_to_card` nested branches.
- Why needed: card system affects movement, money, and jail state.

`test_collect_from_all_action_branches`

- Tests: self player, payer with enough funds, payer without enough funds.
- Branch/path: loop condition true/false paths.
- Why needed: validates multi-player transfer logic.

`test_check_bankruptcy_branches`

- Tests: non-bankrupt path, bankrupt elimination path, non-member path.
- Branch/path: `_check_bankruptcy` branches including index reset condition.
- Why needed: elimination must be safe and consistent.

`test_find_winner_branches_expect_highest_net_worth`

- Tests: no players and non-empty player list.

- Branch/path: both `find_winner` branches.
- Why needed: determines final game outcome.

`test_run_branches_with_and_without_winner`

- Tests: loop execution path + winner ending, and no-player ending.
- Branch/path: `run` loop and end-condition branches.
- Why needed: validates top-level game loop termination paths.

`test_interactive_menu_branches`

- Tests: every menu option, positive/zero loan amount, invalid choice loop-back, roll exit.
- Branch/path: full interactive menu decision tree.
- Why needed: menu is branch-heavy and user-facing.

`test_menu_mortgage_branches`

- Tests: no properties, valid selection, invalid index.
- Branch/path: `_menu_mortgage` branches.
- Why needed: ensures safe interactive indexing.

`test_menu_unmortgage_branches`

- Tests: no mortgaged properties, valid selection, invalid index.
- Branch/path: `_menu_unmortgage` branches.
- Why needed: prevents invalid unmortgage operations.

`test_menu_trade_branches`

- Tests: no opponents, invalid opponent index, no properties, invalid property index, success path.
- Branch/path: full `_menu_trade` decision chain.
- Why needed: trade setup has multiple user-input boundaries.

`test_get_player_names_filters_empty_entries`

- Tests: CSV parsing with blanks and spaces.
- Branch/path: list-comprehension include/skip behavior.
- Why needed: input sanitation prevents empty player names.

`test_main_runs_game_successfully`

- Tests: normal `main()` start path.
- Branch/path: `try` success branch.
- Why needed: verifies standard entry execution.

`test_main_handles_keyboard_interrupt`

- Tests: interruption handling message path.
- Branch/path: `except KeyboardInterrupt` branch.
- Why needed: graceful termination.

`test_main_handles_value_error`

- Tests: setup error handling path.
- Branch/path: `except ValueError` branch.
- Why needed: user feedback on invalid setup.

`test_main_module_entrypoint_branch_executes`

- Tests: script execution via `__main__` guard.
- Branch/path: `if __name__ == "__main__"` true branch.
- Why needed: ensures direct script run works.

`test_property_rent_mortgage_unmortgage_and_availability`

- Tests: rent multiplier, mortgage/unmortgage cycles, availability states.
- Branch/path: all key `Property` decision branches.
- Why needed: property economics are central to gameplay.

`test_property_group_branches_and_owner_counts`

- Tests: duplicate add prevention, add new property, all-owned logic, owner-count include/skip.
- Branch/path: all `PropertyGroup` branches.
- Why needed: group ownership directly controls rent multiplier.

`test_player_money_and_bankruptcy_branches`

- Tests: add/deduct valid and invalid, bankrupt true/false.
- Branch/path: player money validation branches.
- Why needed: guards financial state correctness.

`test_player_move_go_jail_and_property_management`

- Tests: no-go move, pass-go, land-go, go-to-jail, add/remove property branches.
- Branch/path: all movement/property list branches.
- Why needed: movement and ownership are core mechanics.

`test_player_status_line_jail_and_non_jail`

- Tests: status string with/without jailed tag.
- Branch/path: status conditional branch.

- Why needed: UI consistency for player state.

`test_board_lookup_tile_and_ownership_branches`

- Tests: property lookup hit/miss, tile type branches, purchasable branches, ownership filters.
- Branch/path: all `Board` branch paths.
- Why needed: board interpretation drives turn outcomes.

`test_ui_output_and_input_branches`

- Tests: UI print branches for player states and safe input/confirm branches.
- Branch/path: all significant UI condition paths.
- Why needed: verifies terminal interaction behavior for valid and invalid inputs.

4. Step 5: Bugs Found and Fixes

Error 1

- What was wrong: Dice rolled values from 1 to 5 instead of 1 to 6.
- How discovered: `test_dice_roll_updates_streak_and_description` failed (asserted randint bounds).
- Fix applied: changed `random.randint(1, 5)` to `random.randint(1, 6)` for both dice.

Error 2

- What was wrong: Player could not buy property when balance equaled price.
- How discovered: `test_buy_property_branches_and_exact_balance_case` failed.
- Fix applied: changed affordability check from `<=` to `<` in `buy_property`.

Error 3

- What was wrong: Rent was deducted from tenant but not credited to owner.
- How discovered: `test_pay_rent_branches_and_transfer_to_owner` failed.
- Fix applied: added owner balance credit (`prop.owner.add_money(rent)`) in `pay_rent`.

Error 4

- What was wrong: Trade deducted buyer cash but did not pay seller.
- How discovered: `test_trade_branches_and_cash_transfer_to_seller` failed.

- Fix applied: added `seller.add_money(cash_amount)` in `trade`.

Error 5

- What was wrong: Paying jail fine did not deduct money from player.
- How discovered: `test_handle_jail_turn_pay_fine_branch` failed.
- Fix applied: added `player.deduct_money(JAIL_FINE)` in voluntary fine branch.

Error 6

- What was wrong: Winner selection returned minimum net worth player.
- How discovered: `test_find_winner_branches_expect_highest_net_worth` failed.
- Fix applied: replaced `min(...)` with `max(...)` in `find_winner`.

Error 7

- What was wrong: Group ownership check used `any` instead of `all`.
- How discovered: `test_property_group_branches_and_owner_counts` failed.
- Fix applied: changed ownership predicate to `all(...)` in `all_owned_by`.

Error 8

- What was wrong: Passing Go (without landing exactly on 0) did not award salary.
- How discovered: `test_player_move_go_jail_and_property_management` failed.
- Fix applied: updated `Player.move` to award salary when old position + steps crosses board size.

Error 9

- What was wrong: `buy_property` allowed purchases for already-owned properties.
- How discovered: `test_buy_property_rejects_owned_and_mortgaged_cases` failed.
- Fix applied: added ownership guard in `buy_property` to reject already-owned properties.

Error 10

- What was wrong: Failed unmortgage attempts could unintentionally clear the mortgage flag.
- How discovered: updated `test_unmortgage_property_branches` asserted mortgage status after insufficient funds.

- Fix applied: changed `unmortgage_property` to validate affordability before state mutation.

Error 11

- What was wrong: `trade` allowed zero-value trades and raised on negative cash values.
- How discovered: `test_trade_rejects_non_positive_cash_amount` failed.
- Fix applied: added explicit validation requiring trade cash amount to be greater than zero.

Error 12

- What was wrong: Large movement steps only awarded one Go salary even after multiple board wraps.
- How discovered: `test_player_move_large_steps_and_zero_steps` failed.
- Fix applied: updated `Player.move` to compute number of wraps and award salary per pass.

5. Final Coverage Report

Command used:

```
pytest --cov --cov-branch --cov-report=term-missing
```

Final result: - Statement coverage: **100%** - Branch coverage: **100%** - Tests passed: **51/51**

Coverage summary from final run: - `main.py`: 100% - `moneypoly/bank.py`: 100% - `moneypoly/board.py`: 100% - `moneypoly/cards.py`: 100% - `moneypoly/config.py`: 100% - `moneypoly/dice.py`: 100% - `moneypoly/game.py`: 100% - `moneypoly/player.py`: 100% - `moneypoly/property.py`: 100% - `moneypoly/ui.py`: 100%

6. Summary

- Total white-box tests implemented: **51**
- Logical errors fixed: **12**
- Final status: all tests pass and strict **100% branch coverage achieved**.

Additional Coverage.py Validation Round (Latest)

Coverage.py command used:

```
coverage erase
coverage run --branch -m pytest tests
coverage report -m
```

Latest outcome: - Total tests: **60 passed** - Source-code branch coverage: **100%** for all modules in `code/moneypoly/`

Newly Added Test Cases

`test_bank_give_loan_reduces_reserves`

- What it tests: checks that the bank's cash reserves go down after giving a loan.
- Branch/path covered: positive-loan path in `Bank.give_loan` with state update validation.
- Why needed: a loan should reduce bank money; otherwise bank balance becomes unrealistic.

`test_player_move_negative_steps_does_not_collect_go_salary`

- What it tests: moving backwards (negative steps) does not wrongly award Go salary.
- Branch/path covered: unexpected-input path in `Player.move` when `steps <= 0`.
- Why needed: protects against incorrect money gains for invalid or reverse moves.

`test_collect_from_all_allows_exact_balance_payment`

- What it tests: another player pays when their balance is exactly equal to required amount.
- Branch/path covered: equality boundary in `_handle_collect_from_all_card` (`balance >= value`).
- Why needed: exact-boundary values are common failure points in money logic.

`test_bank_collect_ignores_negative_amounts`

- What it tests: verifies that invalid negative amounts do not get treated as incoming bank collections.
- Branch/path covered: negative-input guard path in `Bank.collect`.
- Why needed: protects financial accounting from incorrect reserve reductions via a collect operation.

`test_carddeck_empty_helpers_return_safe_values`

- What it tests: empty deck helper methods for remaining count and debug representation.

- Branch/path covered: empty-list defensive paths in `CardDeck.cards_remaining` and `CardDeck.__repr__`.
- Why needed: empty collections are common edge cases; helper methods should not crash.

`test_player_net_worth_includes_owned_property_values`

- What it tests: verifies that player net worth counts both cash and owned property value.
- Branch/path covered: `Player.net_worth` aggregation path with owned assets.
- Why needed: winner/ranking decisions should reflect total wealth, not cash alone.

`test_play_turn_bankruptcy_does_not_skip_next_player`

- What it tests: when the active player goes bankrupt mid-turn, the next valid player should play next.
- Branch/path covered: `Game.play_turn` post-resolution flow when current player is removed from `self.players`.
- Why needed: this guards turn-order correctness after eliminations.

`test_mortgage_property_handles_insufficient_bank_reserves`

- What it tests: verifies that mortgaging fails safely when bank reserves cannot cover payout.
- Branch/path covered: insufficient-bank-reserve path in `Game.mortgage_property`.
- Why needed: prevents runtime crashes and invalid mortgage state in low-funds scenarios.

`test_trade_rejects_self_trade_action`

- What it tests: ensures a player cannot trade a property with themselves.
- Branch/path covered: seller-equals-buyer guard path in `Game.trade`.
- Why needed: this is an unexpected player action that should be rejected explicitly.

Error 13

- What was wrong: emergency loans were added to player balance, but bank reserves were not reduced.
- How discovered: `test_bank_give_loan_reduces_reserves` failed.
- What was changed: updated `Bank.give_loan` to subtract the loan amount from bank funds before crediting the player.

Error 14

- What was wrong: empty card decks crashed when asking for remaining cards or string representation.
- How discovered: `test_carddeck_empty_helpers_return_safe_values` failed with `ZeroDivisionError`.
- What was changed: added empty-deck guards in `CardDeck.cards_remaining` and `CardDeck.__repr__`.

Error 15

- What was wrong: after a current player was eliminated for bankruptcy, turn advancement could skip the next player.
- How discovered: `test_play_turn_bankruptcy_does_not_skip_next_player` failed (turn moved to `C` instead of `B`).
- What was changed: added a post-resolution check in `Game.play_turn` to end the turn correctly when the active player has been removed.

Error 16

- What was wrong: `Bank.collect` accepted negative values, which incorrectly reduced reserves through an API intended for incoming money.
- How discovered: `test_bank_collect_ignores_negative_amounts` failed.
- What was changed: added a negative-input guard in `Bank.collect` and updated mortgage payout flow to use `Bank.pay_out` instead of negative collection.

Error 17

- What was wrong: player net worth only returned cash balance and ignored owned properties.
- How discovered: `test_player_net_worth_includes_owned_property_values` failed.
- What was changed: updated `Player.net_worth` to include total value of owned properties.

Error 18

- What was wrong: mortgaging a property could crash if the bank had insufficient reserves for payout.
- How discovered: `test_mortgage_property_handles_insufficient_bank_reserves` failed with `ValueError`.
- What was changed: added reserve validation in `Game.mortgage_property` so the action fails cleanly without mutating mortgage state.

Error 19

- What was wrong: the trade flow allowed self-trades (same seller and buyer), which is an invalid action.
- How discovered: `test_trade_rejects_self_trade_action` failed.
- What was changed: added a guard in `Game.trade` to reject self-trades before any transfer logic.

Updated Summary (Including Latest Round)

- Total white-box tests now: **60**
- Total logical errors fixed by tests: **19**
- Final source branch coverage (Coverage.py): **100%**

White-Box Diagram

Diagram file: - `diagrams/Diagram4.jpeg`

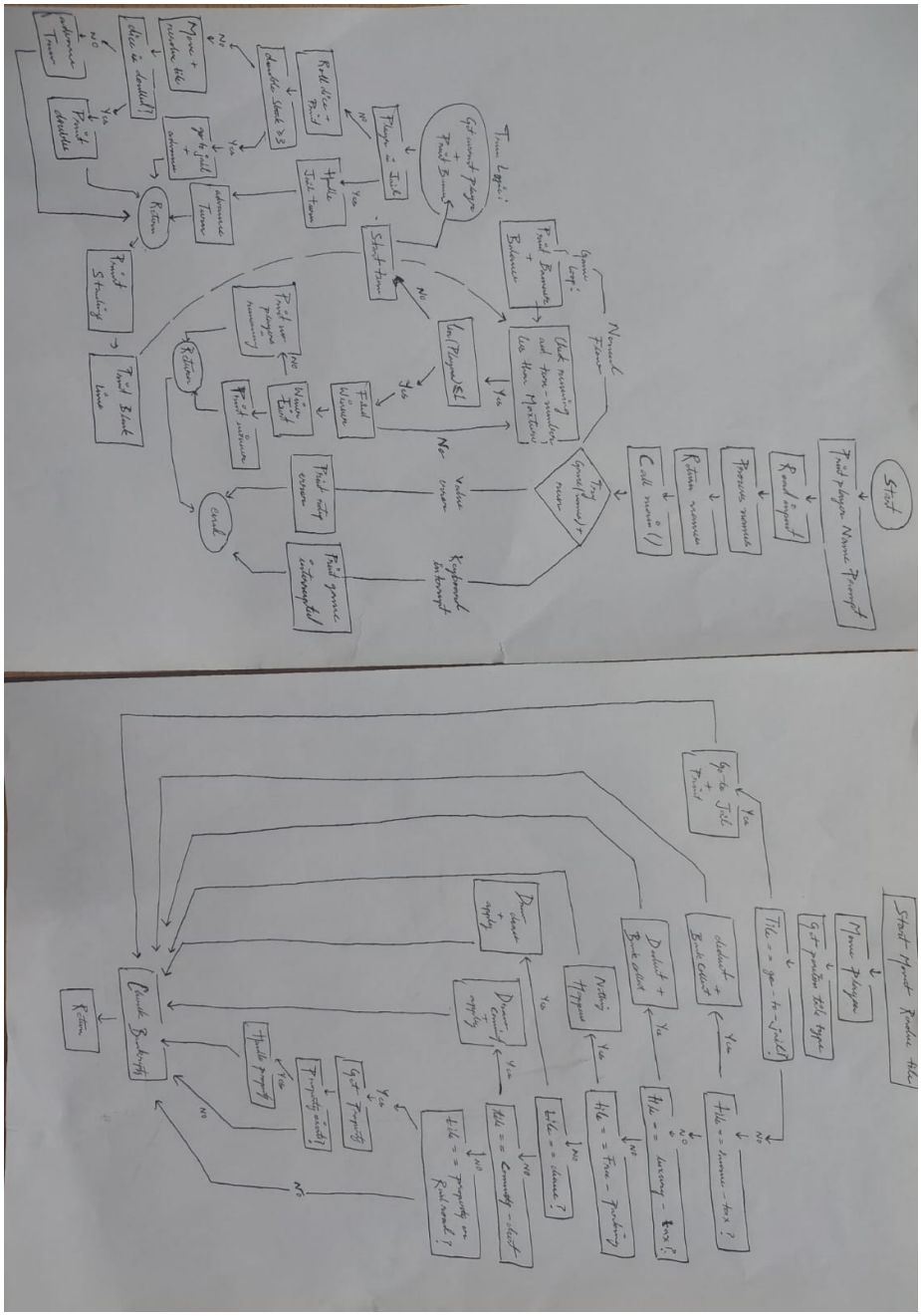

Figure 1: White-Box Call Graph